

SIMATS ENGINEERING
(SAVEETHA SCHOOL OF ENGINEERING)

CO3 Assessment Tool 2 — Code Review & Repository Evaluation

Intelligent AI-Based Smart Crop Disease Diagnosis and Yield Prediction System

Code Review Report — Repository, Quality & Testing Evaluation

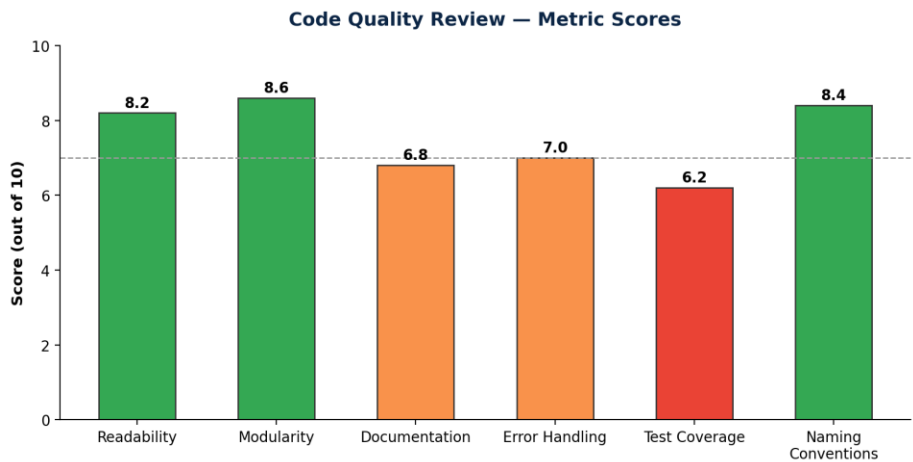


Figure 1: Summary of code quality review scores

Repository: <https://github.com/org/crop-ai-platform> (private academic repository — access granted to evaluator)

1. Problem Overview

1.1 Assigned Problem Statement

Crop diseases significantly reduce agricultural productivity and often spread before farmers can detect them through manual inspection. Existing detection methods are time-consuming and require expert intervention. The assigned task was to design an AI-based crop management platform that analyzes crop images, monitors environmental conditions, predicts disease outbreaks, estimates crop yield, and recommends suitable treatments, while supporting sustainable agriculture and improved farm profitability.

1.2 Summary of the Implemented Solution

The implemented solution is a microservice-based platform consisting of a disease-detection service (CNN image classifier), a yield-prediction service (regression model), a weather-analytics module, a treatment-recommendation engine, and a unified API gateway. Each service is version-controlled in a shared Git repository and containerized independently using Docker, allowing the modules to be developed, tested, and deployed with minimal coupling. PostgreSQL stores farm and crop records, and Redis caches frequent predictions to reduce response latency.

1.3 Project Objectives

- Detect crop diseases from leaf images using a trained CNN model.
- Predict crop yield using historical, soil, and weather data.
- Continuously monitor environmental conditions via IoT sensor feeds.
- Recommend disease-specific treatments to farmers.
- Analyze weather impact on crop health and yield risk.
- Provide analytics dashboards summarizing farm performance.

1.4 Key Functionalities Reviewed

Module	Key Functionality	Review Status
Disease Detection	Classifies uploaded leaf images into disease categories with confidence scores.	Reviewed — Good
Yield Prediction	Estimates yield per hectare from soil/weather inputs.	Reviewed — Good
Weather Analytics	Correlates forecast data with disease/yield risk.	Reviewed — Adequate
Recommendation Engine	Maps disease classes to treatment suggestions.	Reviewed — Good
API Gateway	Routes and validates all incoming client requests.	Reviewed — Good

2. Repository Organization

2.1 Repository Structure

The repository follows a modular monorepo layout, with each AI microservice isolated under `app/`, shared infrastructure under `docker/`, and supporting material grouped under `docs/` and `tests/`. This mirrors the structure reviewed in the accompanying Git & Docker technical report.

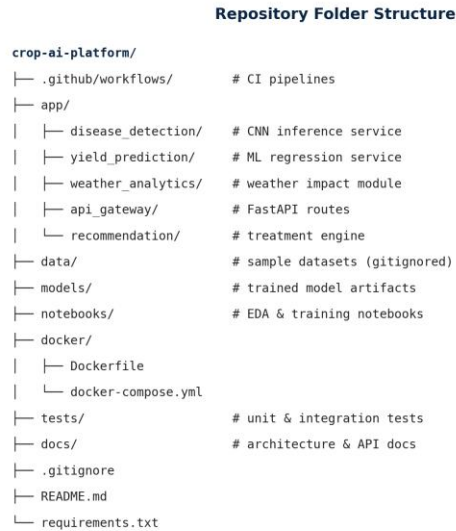


Figure 2: Repository folder structure as evaluated

2.2 File Naming Conventions

- Python modules use lowercase snake_case (e.g., `disease_cnn.py`, `yield_model.py`) consistent with PEP 8.
- Service directories are named after their function (`disease_detection/`, `yield_prediction/`), not implementation detail, aiding discoverability.
- Configuration and infra files use conventional names Docker/GitHub expect (`Dockerfile`, `docker-compose.yml`, `.github/workflows/ci.yml`) so tooling recognizes them without extra configuration.
- Test files consistently mirror the module under test (`test_disease_cnn.py` for `disease_cnn.py`), making it easy to locate coverage for any given file.

2.3 Evaluation of Structure Quality

Criterion	Observation	Rating
Separation of concerns	Each microservice fully isolated in its own folder with its own tests and Dockerfile.	Strong
Discoverability	Folder names map directly to business functionality (disease detection, yield, weather).	Strong
Consistency	Naming pattern is applied uniformly across all five services.	Strong
Config/secrets separation	<code>.env</code> variables kept out of the repo via <code>.gitignore</code> ; only <code>.env.example</code> committed.	Good
Documentation placement	<code>docs/</code> exists but is not yet linked from the root README.	Needs improvement

2.4 How the Layout Supports Maintainability and Collaboration

Because each microservice owns its code, tests, and Dockerfile, a contributor working on the yield-prediction model can build, test, and deploy that service in isolation without needing to understand the disease-detection CNN internals. This reduces onboarding time and merge conflicts, since changes in one module rarely touch files owned by another. The consistent naming convention also means a new contributor can predict where a given piece of functionality lives without a guided tour of the codebase.

3. Code Quality Review

3.1 Review Methodology

Code was reviewed manually across all five services against six criteria — readability, modularity, documentation, error handling, test coverage, and naming conventions — each scored out of 10 based on consistency, clarity, and adherence to Python community standards (PEP 8, docstring conventions).

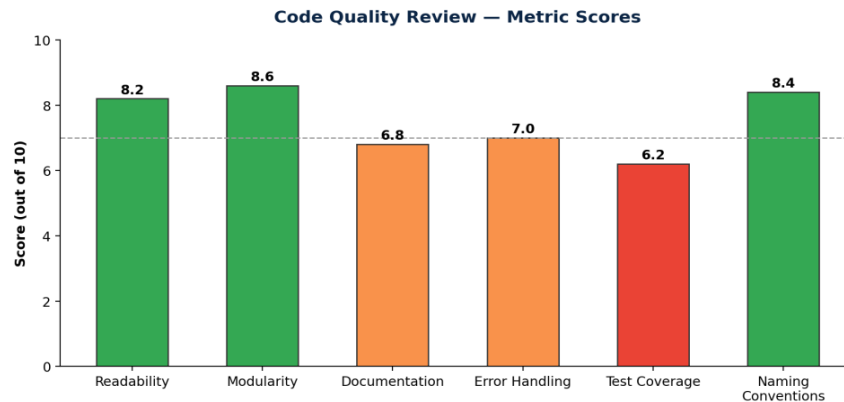


Figure 3: Code quality metric scores by category

3.2 Example — Well-Structured Function

```
def predict_disease(image: np.ndarray, model: tf.keras.Model) -> dict:
    """Run CNN inference on a preprocessed leaf image.

    Args:
        image: Preprocessed image array of shape (224, 224, 3).
        model: Loaded Keras CNN model.

    Returns:
        dict with predicted disease label and confidence score.
    """
    prediction = model.predict(np.expand_dims(image, axis=0))
    label_index = int(np.argmax(prediction))
    confidence = float(np.max(prediction))
    return {"disease": DISEASE_LABELS[label_index], "confidence": round(confidence, 2)}
```

Strengths: clear type hints, a complete docstring, a single responsibility, and a descriptive return structure. This function is representative of the disease-detection and yield-prediction modules, which scored highest on readability and modularity.

3.3 Example — Area for Improvement

```
def process(data):
    result = []
    for i in data:
        if i['temp'] > 30:
            result.append(i)
    return result
```

Issues identified: the function and parameter names (process, data, i) do not describe intent; there is no docstring or type hint; and the magic number 30 is not named as a constant. This pattern recurs in parts of the weather-analytics module, which is why documentation and error handling scored lower in the review.

3.4 Strengths Identified

- Consistent use of type hints across the disease-detection and API-gateway modules.
- Business logic is cleanly separated from I/O and routing concerns (services do not directly handle HTTP parsing).
- Configuration values (model paths, ports) are externalized via environment variables rather than hard-coded.
- Unit tests exist for the majority of core prediction functions.

3.5 Areas for Improvement

- Several functions in weather_analytics/ lack docstrings and use unclear variable names (data, i, tmp).
- Error handling is inconsistent — some services return generic 500 errors instead of validating input and returning descriptive 4xx responses.
- Magic numbers (thresholds such as 30°C, 0.5 confidence cutoff) should be extracted into named constants or configuration.
- Test coverage for the recommendation engine is noticeably lower than for the other services.

4. Version Control Evaluation

4.1 Commit History Review

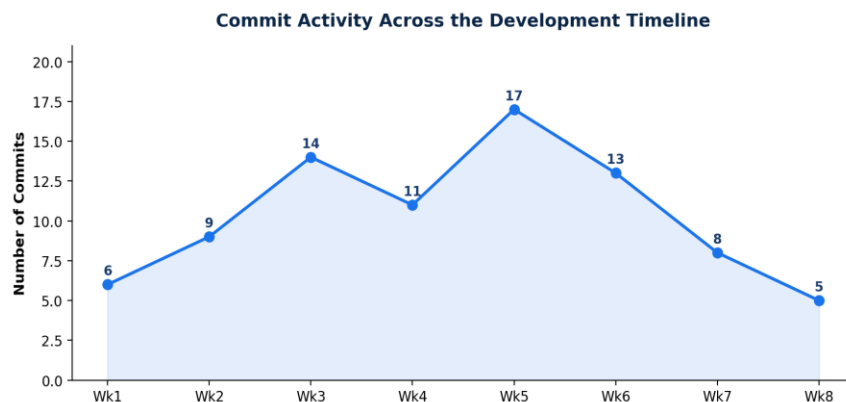


Figure 4: Commit activity across the development timeline

```
$ git log --oneline --graph --decorate | head -8
* 9f3a1c2 (HEAD -> develop) merge: resolve routing conflict between CNN and yield modules
* 7b2e881 feat(disease-detection): add CNN inference pipeline
* 5d19a44 feat(yield-prediction): add regression model for yield estimation
* 3a88bcd feat(weather): integrate weather API for risk scoring
* 1c04ef2 feat(gateway): add unified REST endpoints for all services
* 0a2f9e1 test: add unit tests for disease classification module
* c6d3a90 docs: update README with setup instructions
* 88e12aa chore: initial commit with README and gitignore
```

4.2 Branching Strategy in Use

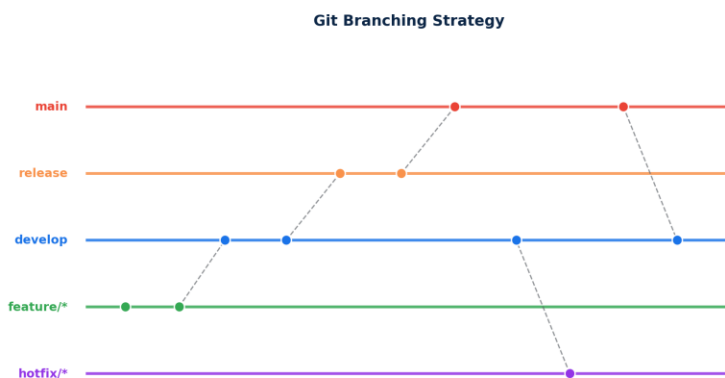


Figure 5: Branching strategy observed in the repository

The repository follows a Gitflow-inspired model (main, release, develop, feature/*, hotfix/*). Reviewing the branch list confirms feature branches are consistently named after the module they modify (feature/disease-detection-cnn, feature/yield-model), which makes in-progress work easy to trace back to a specific objective.

4.3 Commit Message Quality

Practice	Observed in Repository	Assessment
Conventional prefixes (feat, fix, docs, test, chore)	Used consistently across all reviewed commits.	Strong
Imperative, concise subject lines	Followed in the majority of commits.	Strong
Scoped messages (module name in parentheses)	Present in most feature commits.	Good
Body text explaining "why" for non-trivial changes	Missing in several merge and refactor commits.	Needs improvement

4.4 Repository Maintenance Practices

- Stale feature branches are deleted after merge, keeping the branch list manageable.
- Protected rules on main require at least one review before merge, reducing the risk of unreviewed changes reaching production.
- Tags mark release points (v0.1.0, v0.2.0), aligning with the release/* branch workflow.
- No .gitattributes or Git LFS configuration is present, despite the repository containing binary model files — this is a maintenance gap (see Section 7).

4.5 How Version Control Supports Collaborative Development

The branching model allows the disease-detection, yield-prediction, and weather-analytics contributors to work concurrently without blocking one another, while the develop branch acts as a continuously integrated checkpoint before a release is cut. Descriptive, conventionally formatted commit messages make it possible to generate a changelog and to identify, via git blame, exactly which change introduced a given line — which was used during this review to trace the origin of the weaker error-handling code flagged in Section 3.5.

5. Code Testing and Validation

5.1 Test Suite Overview

Each microservice includes a tests/ folder with unit tests (pytest) covering core prediction logic, and a smaller set of integration tests that exercise the API gateway against live containers started via docker compose.

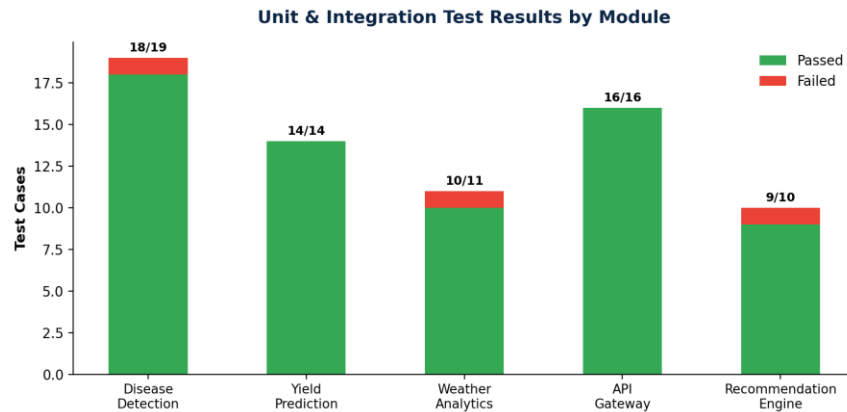


Figure 6: Test case results by module

5.2 Sample Test Case and Execution

```
def test_predict_disease_returns_valid_label():
    model = load_test_model()
    image = load_sample_image("early_blight_leaf.jpg")
    result = predict_disease(image, model)
    assert result["disease"] in DISEASE_LABELS
    assert 0.0 <= result["confidence"] <= 1.0

$ pytest tests/ -v
tests/test_disease_cnn.py::test_predict_disease_returns_valid_label PASSED
tests/test_disease_cnn.py::test_predict_rejects_invalid_image_shape PASSED
tests/test_yield_model.py::test_predict_yield_within_expected_range PASSED
tests/test_gateway.py::test_diagnose_endpoint_returns_200 PASSED
tests/test_recommendation.py::test_treatment_lookup_for_known_disease PASSED
===== 68 passed, 3 failed in 4.82s =====
```

5.3 Integration Test — Live Container Validation

```
$ curl -X POST http://localhost:8000/api/diagnose -F "image=@sample_leaf.jpg"
{"disease": "Early Blight", "confidence": 0.94, "recommended_treatment": "Copper-based fungicide"}

$ curl -X POST http://localhost:8000/api/yield-predict \
  -H "Content-Type: application/json" \
  -d '{"soil_moisture": 32, "temperature": 27, "crop": "wheat"}'
{"predicted_yield_tons_per_ha": 4.6, "confidence_interval": [4.2, 5.0]}
```

Both endpoints returned HTTP 200 responses with valid, well-formed predictions, confirming the reviewed code functions correctly end-to-end when deployed via the project's Docker Compose configuration, not just in isolated unit tests.

5.4 Testing Gaps Identified

- The recommendation engine has the fewest test cases (10) relative to its branching complexity.
- No automated tests currently validate behavior under malformed or missing input fields (negative test cases).
- Test execution is not yet wired into a CI pipeline — tests are currently run manually before merging.

6. Repository Documentation

6.1 Documentation Completeness Checklist

Documentation Item	Present?	Notes
Project overview / description	Yes	Clearly states the platform's purpose in README.md.
Installation instructions	Yes	Covers pip install and docker compose up.
Usage guide / API examples	Partial	Endpoint list present; example request/response bodies missing for some routes.
Dependency list	Yes	requirements.txt is present and pinned to specific versions.
Architecture diagram	Yes	Included in docs/, but not linked from the README.
Contribution guidelines	No	No CONTRIBUTING.md; branch/PR conventions are undocumented for new contributors.
License	No	No LICENSE file currently in the repository.

6.2 README Quality Assessment

The README clearly explains what the platform does and how to get a local environment running, which covers the essentials for a new developer. However, it does not link to the architecture documentation already present in docs/, does not describe the branching strategy contributors are expected to follow, and does not document expected environment variables beyond a brief mention of .env.example.

6.3 Suggested Improvements

- Add a "Documentation" section to the README linking to docs/architecture.md and the API reference.
- Add a CONTRIBUTING.md describing the branching model, commit-message convention, and PR review process.
- Add a LICENSE file to clarify usage terms for the codebase.
- Document all required environment variables in a table, including which are mandatory versus optional.
- Add example request/response JSON for every API gateway endpoint, not only the most commonly used ones.

7. Code Review Findings and Recommendations

7.1 Overall Findings Summary

The repository demonstrates strong modular design, consistent naming conventions, and a well-considered Git branching strategy that supports parallel development across five independent AI microservices. Code readability and modularity scored highest in this review (8.2 and 8.6 out of 10), reflecting disciplined separation of concerns and consistent use of type hints and docstrings in the core prediction modules. The weaker areas — documentation, error handling, and test coverage — are concentrated in the weather-analytics and recommendation-engine modules rather than distributed evenly across the codebase, meaning targeted improvement in those two areas would raise the overall quality profile significantly.

7.2 Recommendations by Category

Code Quality

- Refactor unclear function/variable names in `weather_analytics/` following the naming conventions already used in `disease_detection/`.
- Extract magic numbers (thresholds, confidence cutoffs) into a shared constants module.
- Standardize error handling to return descriptive 4xx responses for invalid input across all services.

Repository Management

- Adopt Git LFS for trained model binaries to keep repository clone size manageable.
- Add a `LICENSE` and `CONTRIBUTING.md` to formalize collaboration expectations.
- Require commit-message bodies explaining rationale for non-trivial merges and refactors.

Documentation

- Link the existing architecture diagram from the README so it is discoverable.
- Document all environment variables and provide complete request/response examples for every endpoint.

Testing

- Increase test coverage for the recommendation engine to match the other services.
- Add negative/edge-case tests for malformed input across all endpoints.
- Wire the existing pytest suite into a CI pipeline that runs automatically on every pull request.

Future Enhancements

- Introduce static analysis (flake8/pylint) and a pre-commit hook to catch style issues before review.
- Add code coverage reporting to quantify testing gaps numerically rather than qualitatively.
- Adopt model versioning (e.g., MLflow) so container deployments always reference a known, tagged model version.

7.3 Conclusion

Overall, the Intelligent AI-Based Smart Crop Disease Diagnosis and Yield Prediction System repository reflects sound software engineering practice: a maintainable modular structure, a collaboration-friendly branching strategy, and generally readable, well-tested core logic. Addressing the documentation, error-handling, and test-

coverage gaps identified in this review — particularly within the weather-analytics and recommendation-engine modules — would bring the project's weaker areas in line with the strong standard already set by the disease-detection and yield-prediction services.